4a Process Synchronization

(some recap from CSC2002S)



Michelle Kuttel

Professor

Department of Computer Science

University of Cape Town

"Your obligation is that of active participation. You should not act as knowledge-absorbing sponges, but as whetstones on which we can all sharpen our wits"
— **Edsger W. Dijkstra**

Concurrency and synchronization problems

Concurrent access to shared data is prone to **data races** and other issues of data integrity - **data inconsistency.**

- **race condition:** correctness depends on timing/interleaving
- **data race:** unsynchronised concurrent access to the same memory location, with at least one write

Maintaining data consistency requires **mechanisms** to ensure the **orderly execution** of cooperating processes.

The synchronization mechanism is usually provided by **both hardware** and **the operating system.**

- The ability to execute an instruction, or a number of instructions, **atomically** is crucial for being able to solve many of the synchronization problems.

> "If debugging is the process of removing software bugs, then programming must be the process of putting them in."
> — **Edsger W. Dijkstra**

Operating Systems are highly concurrent

Two events are concurrent if we ***cannot tell*** *which will* ***happen first.***

Multiple *threads/processes*

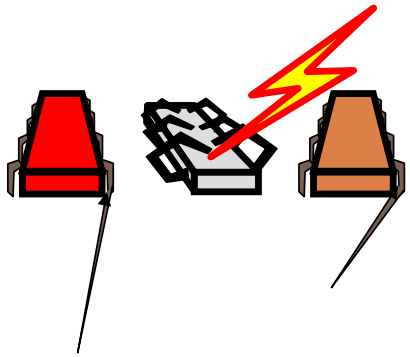
Single shared *memory for some*

Unpredictable asynchronous delays

- arbitrary speed for each thread/process

Interleaving

- arbitrary order for threads/processes



Delays from:

- Cache misses (*short*)
- Page faults (*long*)
- Scheduling quantum used up (*really long*)



Synchronization -reminder

Synchronization constraints are **requirements** pertaining to the **order of events.**

- **E.g**
- **Mutual exclusion**: Event A and Event B must **not** happen ***at the same time.***
  - Covered in CSC2002S

This is the programmer's responsibility

- "The compiler" has no idea what interleavings should or shouldn't be allowed in your program
- you need language primitives to do it!



Also - issues with synchronization: **Liveness, starvation** and **deadlock**

Liveness – processes must make **progress**, not wait indefinitely.

Deadlock

- Two or more threads are permanently waiting for each other
- None of them can make progress

Starvation

- A thread waits indefinitely because others keep getting the resource
- Some threads make progress, but one is continually denied access

Livelock

- Threads keep changing state in response to each other
- They are active, but no useful progress is made
- Key distinction

Deadlock: stuck and inactive
Starvation: one thread never gets a turn
Livelock: active but still stuck

Do Morning Quiz 7 now

- Password: priority

Basic synchronization patterns

Constraints on order

- **Signaling**
  - a1 before b1

Thread A
```
statement a1
```

Thread B
```
statement b1
```

- **Rendezvous**
  - a1 before b2 **and** b1 before a2.

Thread A
```
statement a1
statement a2
```

Thread B
```
statement b1
statement b2
```

- **Barrier**
  - All threads must wait until all arrive
- **Reusable Barrier**

Constraints on access

- **Mutual exclusion (Mutex)**
  - Critical section cannot be executed simultaneously - one thread at a time

Thread A
```
critical section
```

Thread B
```
critical section
```

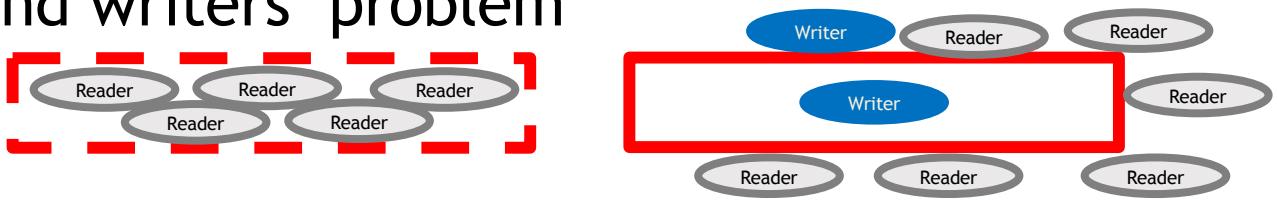
- **Multiplex**
  - critical section with ***n*** threads allowed at a time

Classical synchronization problems/ patterns/idioms

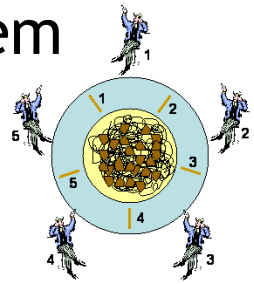
- Producer-consumer (or bounded buffer) problem



- Readers and writers problem



- Dining philosophers problem



These problems are analogous to common problems that operating systems (and some applications) need to solve.

Classical synchronization problems: Producer-consumer problem



In multithreaded programs there is often a **division of labour** between threads.

In one common pattern, some threads are **producers** -

- create items of some kind and add them to a data structure;

and some are **consumers** -

- remove the items and process them.

e.g. Event-driven programs.

- Whenever an event occurs, a **producer thread** creates an **event object** and adds it to the **event buffer**.
- Concurrently, **consumer threads** ("event handlers") take events out of the buffer and process them

Classical synchronization problems: Producer-consumer problem



In multithreaded programs there is often a **division of labour** between threads.

In one common pattern, some threads are **producers** -

- create items of some kind and add them to a data structure;

and some are **consumers** -

- remove the items and process them.


consumer

e.g. Event-driven programs.

- Whenever an event occurs, a **producer thread** creates an **event object** and adds it to the **event buffer**.
- Concurrently, **consumer threads** ("event handlers") take events out of the buffer and process them

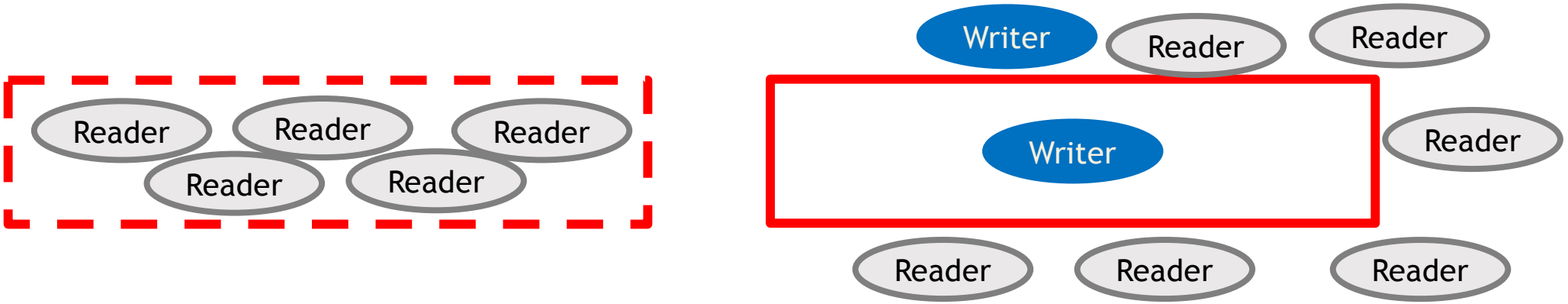
Classical synchronization problems: Producer-consumer problem cont.



Constraints to be enforced by synchronization:

- **Shared buffer** must be **protected**:
  - While an item is being added to or removed from the buffer, the buffer is in an inconsistent state. Therefore, threads must have mutually exclusive access to the buffer.
- Can't **consume from an empty buffer**:
  - If a consumer thread arrives while the buffer is empty, it blocks until a producer adds a new item.
- In *bounded buffer* variant, **can't add to a full buffer**:
  - producer cannot produce if buffer is full.

Classical synchronization problems: Readers -Writers problem



Any situation where a data structure, database, or file system is read and modified by concurrent threads, some of which *read* and others *write*.

Synchronization constraints: *"Categorical mutual exclusion"*

- **Many readers can read concurrently**
  - **Any number of readers** can be in the **critical section** simultaneously.
- **Writers must have exclusive access** to the **critical section**.

Several variants of how readers and writers are considered – all involve priority.

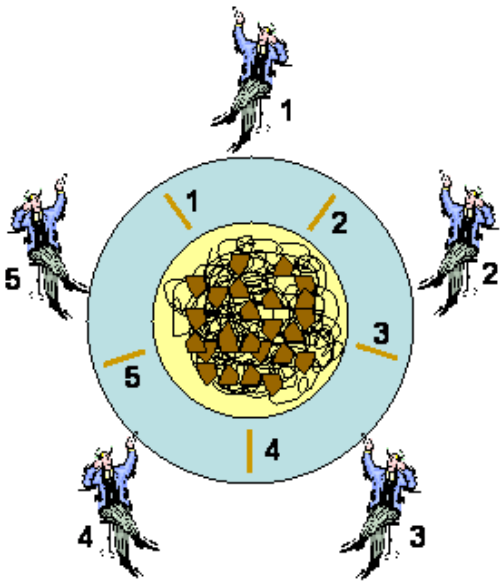
- ***reader-preference*** – no reader kept waiting unless writer has permission to use shared object
- ***writer-preference*** – once writer is ready, it performs the write ASAP
- ***fair / no-starvation***

Classical concurrency problems: Dining philosophers

Classic problem used to illustrate deadlock
- proposed by Dijkstra in 1965

- a table with **five silent philosophers**, **five plates**, **five chopsticks** and a big bowl of rice.
- Each philosopher must alternately **think** and **eat**.
- Eating is not limited by the amount of food left: assume an infinite supply.
- However, a **philosopher needs two chopsticks** to eat
- A chopstick is placed between each pair of adjacent philosophers.

- Standard solutions **prone to deadlock**



unrealistic, ***unsanitary*** *and* ***interesting***

Synchronization primitives used for solutions to synchronization problems

Techniques range from hardware to software-based API's.

Hardware: atomic hardware instructions; atomic variables

Software: mutexes, semaphores, monitors and condition variables, concurrent data structures.

Hardware solutions to synchronization
1. `test-and-set()`

Hardware provides special **atomic** **hardware instructions** to implement locks, and so solve the **critical section problem**.

`test-and-set()`

Combines the actions of **testing variable** and **setting it to a specific value** into a **single machine instruction** which cannot be interrupted

```
boolean test_and_set (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```

Because it is implemented in hardware, it is guaranteed to be atomic.

*pseudocode shows the **effect** of an atomic instruction*

"Grab this flag and mark it occupied."

Hardware solutions to synchronization
1. `test-and-set()`

Hardware provides special **atomic** **hardware instructions** to implement locks. Two types.

`test-and-set()`

Combines the actions of **testing variable** and **setting it to a specific value** into a **single machine instruction** which cannot be interrupted

```
boolean test_and_set (boolean *target)
       {
           boolean rv = *target;
           *target = TRUE;
           return rv;
        }
```

Critical section problem solved with hardware `test-and-set()`

```
do {
        while (test_and_set(&lock)); /* do nothing */

        /* critical section */

        lock = false;

        /* remainder code */

} while (true);
```

*This loop spins until the return value is false, meaning the lock was false, but is now **true** and held.*

Set the lock to false after the critical section, so another thread can acquire it..

a simple primitive for basic mutual exclusion



Hardware solutions to synchronization

2.compare-and-swap()

Hardware provides special **atomic hardware instructions** to implement locks. Second type

`compare-and-swap()`

Swaps the value of a variable only if it is equal to a given expected value.

```
int compare_and_swap(int *value, int expected, int new_value)
   {
         int temp = *value;

         if (*value = = expected)
              *value = new_value;
         return temp;
     }
```

"Change this value, but only if it still has the value I expect."

Hardware solutions to synchronization

2.compare-and-swap()

Hardware provides special **atomic hardware instructions** to implement locks.

`compare-and-swap()`

Swaps the value of a variable only if it is equal to a given expected value.

```
int compare_and_swap(int *value, int
expected, int new_value)
   {
        int temp = *value;

        if (*value == expected)
            *value = new_value;
        return temp;
    }
```

Critical section problem solved with hardware `compare-and-swap()`

This loop spins while the returned old value is not 0.
- *if CAS returns 0, the lock was free and you acquired it*
- *if it returns 1, the lock was already held*

```
while (true) {
  while (compare and swap(&lock, 0, 1) != 0)
  ; /* do nothing */
  /* critical section */
  lock = 0;          Release the lock
  /* remainder section */
}
```

more general primitive that supports more advanced concurrent algorithms



Do Morning Quiz 8 now

- Password: lock

Hardware solutions to synchronization

2.compare-and-swap()

Hardware provides special **atomic hardware instructions** to implement locks.

`compare-and-swap()`

Swaps the value of a variable only if it is equal to a given expected value.

```
int compare_and_swap(int *value, int
expected, int new_value)
   {
        int temp = *value;

        if (*value == expected)
            *value = new_value;
        return temp;
     }
```

Signalling problem solved with hardware `compare-and-swap()`

```
shared int flag = 0

procedure signal()
        compare_and_swap(flag, 0, 1)
end procedure

procedure wait()
        while flag == 0 do
        ; // busy wait
        end while
end procedure
```

more general primitive that supports more advanced concurrent algorithms



Atomic variables

```
int compare_and_swap(int *value, int
expected, int new_value)
   {
          int temp = *value;

          if (*value == expected)
              *value = new_value;
          return temp;
   }
```

The `compare-and-swap()` hardware not usually used *directly* to provide mutual exclusion, but rather

used **to construct** synchronization tools such as **atomic variables.**

Updates to atomic variables are **guaranteed** to be atomic – no data races on a single variable.

```
void increment(atomic int *v)
{
       int temp;
       do {
              temp = *v;
       }
       while (temp != compare and swap(v, temp, temp+1));
}
```

Atomic variables solve atomic update of a single variable; they do not by themselves protect compound invariants across multiple variables.

Atomic variables

The `compare-and-swap()` hardware not usually used *directly* to provide mutual exclusion, but rather

used to construct synchronization tools such as **atomic variables**.

Updates to atomic variables are **guaranteed** to be atomic – no data races on a single variable.

```
void increment(atomic int *v)
{
        int temp;
        do {
                temp = *v;
        }
        while (temp != compare and swap(v, temp, temp+1));
}
```

Hardware solutions are complicated and generally inaccessible to application programmers, so OS designers build software tools to solve critical section problem.

Software solutions to synchronization: mutex lock

Simplest is the **mutex lock**, which has an associated Boolean variable "`available`" to indicate if the lock is available or not.

mutex is short for mutual exclusion

- Two operations to access : `acquire()` and `release()`

```
acquire() {
        while (!available);
            /* busy wait */
        available = false;
    }
```

```
release() {
        available = true;
    }
```

For critical section problem:

```
do {
     acquire lock
       critical section
     release lock
       remainder section
   } while (true);
```

Calls to **`acquire()`** and **`release()`** are atomic: the pseudocode is conceptual, real implementations rely on hardware atomic instructions

Solution requires **busy waiting**: lock is therefore called a **spinlock**.

Software solutions to synchronization: mutex lock

Simplest is the **mutex lock**, which has an associated Boolean variable "`available`" to indicate if the lock is available or not.

- Two operations to access : `acquire()` and `release()`

```
acquire() {
        while (!available);
            /* busy wait */
        available = false;
    }
```

```
release() {
        available = true;
    }
```

For critical section problem:

```
do {
     acquire lock
       critical section
     release lock
       remainder section
   } while (true);
```

Calls to **`acquire()`** and **`release()`** are atomic: usually implemented via hardware atomic instructions.

Solution requires **busy waiting**: lock is therefore called a **spinlock**.

Why is **busy waiting** a disadvantage? What are the **advantages?**



Spin locks in general

"**Busy waiting**" - process repeatedly checks the status of variable to determine whether or not an event has occurred.

Disadvantage: wastes CPU cycles

Advantage **no context switch is required**

- waiting on a lock requires **two context switches**—
  - a context switch to move the thread to the waiting state
  - a second context switch to restore the waiting thread once the lock becomes available

Spinlocks are locking mechanism of choice on multiprocessor systems if the lock will be held for a duration of **less than two context switches.**

- on multicore systems, one thread can “spin” on one processing core while another thread performs its critical section on another core.

Can you implement your own mutex lock?



- Example with a Boolean available
- Try with 5 threads
- Try with 25 threads

`BoolTest`

Lock contention

A lock is **contended** if a thread **blocks** while trying to acquire the lock.

In **high contention** a relatively large number of threads attempt to acquire the lock.

Highly contended locks tend to **decrease overall performance** of concurrent applications

Do Morning Quiz 9 now

- Password: rain



Edsgar Dijkstra

Software solutions to synchronization: semaphores



More robust and sophisticated synchronization tool (than mutex locks).

Semaphore ***S*** - integer variable accessed **only** via two **atomic operations:**

- `wait()` and `signal()`
  - Originally called **`P()`** and **`V()`**

In Java, called `acquire( )` and `release()`

```
wait(S) {
        while (S <= 0)
          ; // busy wait
        S = S - 1;
    }
```

```
signal(S) {
        S = S + 1;
    }
```

For critical section problem:

Create a semaphore "**`synch`**" initialized to 1

```
wait(synch)
    critical section
signal(synch);
```

abstract semaphore operations are **wait** and **signal**

- naive pseudocode may spin but
- practical OS / Java semaphore implementations usually block waiting threads

Types of semaphore

- **Binary semaphore** – integer value can range only between 0 and 1
  - same as a **mutex lock**
- **Counting semaphore** – integer value can range over an unrestricted domain.
  - Used to control access to **a restricted number of processes**

```
wait(S) {
        while (S <= 0)
          ; // busy wait
        S = S - 1;
    }
```

```
signal(S) {
        S = S + 1;
    }
```

Multiplex

e.g. To allow at **most two** processes to execute in a critical section :

Create a semaphore "**synch**" **initialized to 2**

```
wait(synch)
    critical section
signal(synch);
```

Semaphore

- **positive** value = number of permits available
- zero = no permits available. Threads calling wait/acquire when zero **will block**

Software solutions to synchronization: semaphores

When you create **a counting semaphore**, you can **initialize its value to any integer**, but after that the only operations you are allowed to perform are **increment** and **decrement**.

- **decrement**.
  - When a thread decrements the semaphore, if the result is negative, the thread blocks itself and cannot continue until another thread increments the semaphore.
- **increment**
  - When a thread increments the semaphore, if there are other threads waiting, one of the waiting threads gets unblocked.

In the original definition you **cannot read the current value of the semaphore**.

- Although Java will let you do this.

Semaphore implementation

Must **guarantee** that no two processes can execute the **`wait()`** and **`signal()`** on the same semaphore at the same time – usually done with hardware locks.

Also, want to avoid **busy waiting**

(although this is not a problem if critical section, and hence waiting time, is short)

- Modify **`wait()`** so that process blocks (suspends) and is put in ready queue, CPU scheduler chooses another process to execute.
- Each semaphore has an associated *waiting queue*
  - Each entry in a *waiting queue* has two data items:
    - value (of type integer);
    - pointer to next record in the list

```
typedef struct{
            int value;
            struct process *list;
} semaphore;
```

Two operations:

- **block ()** – place the process invoking the operation on the appropriate waiting queue
- **wakeup (P)** – remove one of processes in the waiting queue and place it in the ready queue

Java Semaphores

- Constructor: `Semaphore(int value);`

Semaphore(int permits)	Creates a Semaphore with the given number of permits and nonfair fairness setting.
Semaphore(int permits, boolean fair)	Creates a Semaphore with the given number of permits and the given fairness setting. If fair true, thread waiting longest gets the semaphore permit

- Usage:

```
Semaphore sem = new Semaphore(1);

try {
   sem.acquire();
   /* critical section */
}
catch (InterruptedException ie) { }
finally {
   sem.release();
}
```



Basic synchronization patterns solution with semaphores

- **Signaling**
  - a1 before b1

Semaphore "**sem**" **initialized to 0**

Thread A

```
statement a1
sem.signal ()
```

Thread B

```
sem.wait ()
statement b1
```

Note that what a semaphore is initialized to matters!

- **Rendezvous**
  - a1 before b2 **and** b1 before a2.

Both semaphores **initialized to 0**

Thread A

```
statement a1
aArrived.signal()
bArrived.wait()
statement a2
```

Thread B

```
statement b1
bArrived.signal()
aArrived.wait()
statement b2
```

- **Barrier**
  - All threads must wait at barrier until all arrive

Semaphore "**barrier**" **initialized to -(n-1)**

All threads

```
barrier.signal()
barrier.wait()
barrier.signal()
```

"turnstile" solution

Producer

Consumer




```
while (true) {
  /* produce item in next_produced */

  while (counter == BUFFER_SIZE) ;
       /* full buffer- do nothing */

   buffer[in] = next_produced;
   in = (in + 1) % BUFFER_SIZE;
   counter++;
}
```

```
while (true) {
while (counter == 0) ;
       /*empty buffer -  do nothing */

       next_consumed = buffer[out];
       out = (out + 1) % BUFFER_SIZE;

       counter--;
  /* consume item in next=_consumed */
}
```

`buffer` is implemented as a **circular array**, with two logical pointers, `in` and `out`:
`in` points to **next free position** in buffer,
`out` points to **first full position.**
Buffer is empty when `in=out` and full when `(in+1)%BUFFER_SIZE=out`.

Producer

Consumer




```
while (true) {
  /* produce item in next_produced */

  while (counter == BUFFER_SIZE) ;
       /* full buffer- do nothing */

   buffer[in] = next_produced;
   in = (in + 1) % BUFFER_SIZE;
   counter++;
}
```

```
while (true) {
while (counter == 0) ;
       /*empty buffer -  do nothing */

       next_consumed = buffer[out];
       out = (out + 1) % BUFFER_SIZE;

       counter--;
  /* consume item in next=_consumed */
}
```

May not function correctly when executed concurrently - **race condition**

Producer  Consumer

solution with semaphores

```
do {
        ...
    /* produce item in next_produced */
        ...
     wait(empty);
     wait(mutex);
        ...
    /* add next_produced to buffer */
        ...
     signal(mutex);
     signal(full);
} while (true);
```

```
do {
     wait(full);
     wait(mutex);
        ...
/* remove item from buffer to next_consumed */
        ...
     signal(mutex);
     signal(empty);
        ...
    /* consume item in next_consumed */
        ...
} while (true);
```

Shared variables:

- int **n** ;
- Semaphore **mutex** = 1
- Semaphore **full** = 0
- Semaphore **empty** = n

*What is the purpose of **each** of these shared variables?*



Producer  Consumer

solution with semaphores

```
do {
        ...
    /* produce item in next_produced */
        ...
     wait(empty);
     wait(mutex);
        ...
    /* add next_produced to buffer */
        ...
     signal(mutex);
     signal(full);
} while (true);
```

```
do {
     wait(full);
     wait(mutex);
        ...
/* remove item from buffer to next_consumed */
        ...
     signal(mutex);
     signal(empty);
        ...
    /* consume item in next_consumed */
        ...
} while (true);
```

Shared variables:

- int **n** ;
- Semaphore **mutex** = 1
- Semaphore **full** = 0
- Semaphore **empty** = n

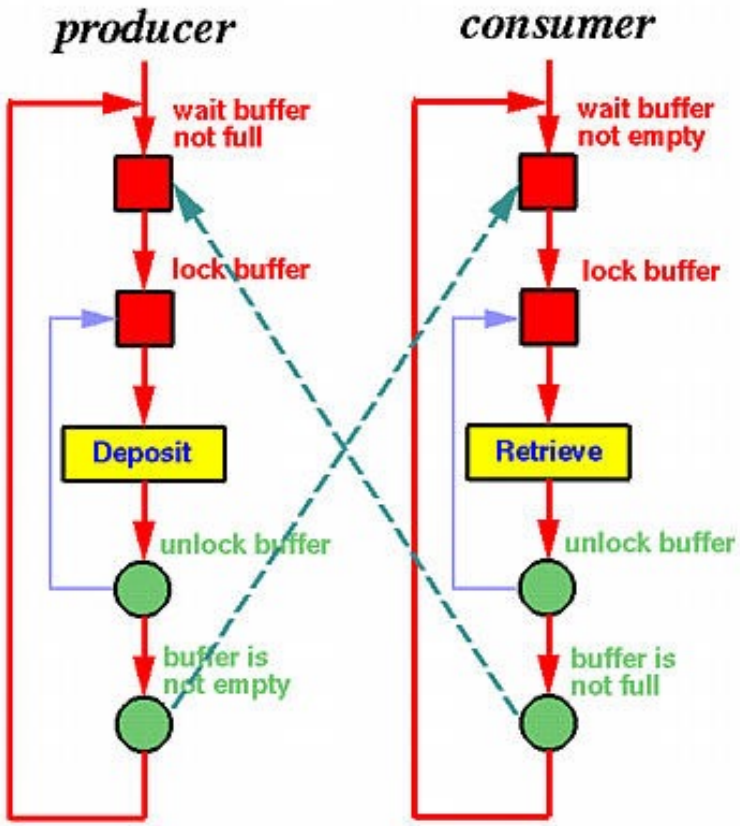
mutex — *mutual exclusion on buffer*
full — *number of full slots*
empty — *number of empty slots*



solution with semaphores

Producer

```
do {
        ...
        /* produce item in
next_produced */
        ...
        wait(empty);
        wait(mutex);
        ...
        /* add next_produce
buffer */
        ...
        signal(mutex);
        signal(full);
    } while (true);
```



Consumer

```
do {
        wait(full);
        wait(mutex);
        ...
/* remove item from buffer to
next_consumed */
        ...
        signal(mutex);
        signal(empty);
        ...
        /* consume item in
next_consumed */
        ...
    } while (true);
```

Producer-Consumer dishwashing with semaphores



- DishWashing_ProducerConsumer_semaphor.zip

```
package dishWashS;
import java.util.concurrent.Semaphore;


public class WetDishRack {
private Semaphore mutex; // can I access rack?
private Semaphore emptySpace; // does the rack have
empty spaces?
private Semaphore dishesExist;  // does the rack
contain dishes?
private int[] rack;
private int in, out;
private int RACK_SIZE;

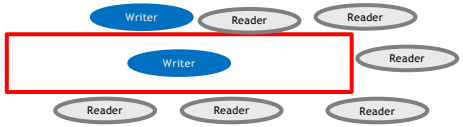

WetDishRack(int rackSize) {
    this.mutex = new Semaphore(1); //
    this.emptySpace = new Semaphore(rackSize); //
    this.dishesExist = new Semaphore(0); //
    this.rack = new int[rackSize]; //
    this.in = 0;
    this.out =0;
    this.RACK_SIZE=rackSize;
}
```

```
public void addDish(int dish_id)  throws
InterruptedException {
    emptySpace.acquire(); //wait until empty space
    mutex.acquire(); //acquire rack
    rack[in]=dish_id; // add dish to rack
    in=(in+1)%RACK_SIZE; //circular buffer
    mutex.release(); //release rack
    dishesExist.release(); //add a permit to full
}


public int removeDish() throws InterruptedException
{
    dishesExist.acquire(); //wait until item in
    array
    mutex.acquire(); //acquire rack
    int next=rack[out];
    out = (out + 1) % RACK_SIZE; //circular buffer
    mutex.release(); //release rack
    emptySpace.release(); //add a permit to empty
    return next;
}
}
```

Java's `Semaphore` is a library abstraction over lower-level concurrency mechanisms

Readers -Writers problem solution with semaphores



Writer

```
do {
        wait(rw_mutex);

            ...
        /* writing */

            ...

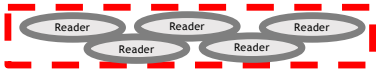
        signal(rw_mutex);
    } while (true);
```

shared data:

```
semaphore rw_mutex = 1
semaphore mutex =1
int read_count = 0
```

Solutions may have **starvation**.

Reader

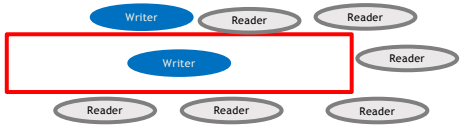



```
do {
        wait(mutex);
        read_count++;
        if (read_count == 1)
            wait(rw_mutex);
        signal(mutex);

        /* reading */

        wait(mutex);
        read_count--;
        if (read_count == 0)
            signal(rw_mutex);
        signal(mutex);
    } while (true);
```

Readers -Writers problem solution with semaphores



Writer

```
do {
        wait(rw_mutex);

            ...
        /* writing */

            ...

        signal(rw_mutex);
    } while (true);
```

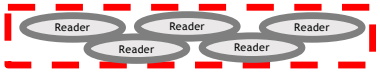
shared data:

semaphore **rw_mutex** = 1
semaphore mutex =1
int read_count = 0

reader-preference solution allows concurrent readers

- writer exclusion enforced
- but writers may starve

Reader




```
do {
        wait(mutex);
        read_count++;
        if (read_count == 1)
            wait(rw_mutex);
        signal(mutex);

        /* reading */

        wait(mutex);
        read_count--;
        if (read_count == 0)
            signal(rw_mutex);
        signal(mutex);

    } while (true);
```

Lightswitch pattern

Readers-Writers pool cleaning with semaphores



```
import java.util.concurrent.Semaphore;


public class Pool {
private Semaphore swim;
private Semaphore clean;
private int numberSwimmers;


Pool() {
    this.swim = new Semaphore(1);
    this.clean = new Semaphore(1);
this.numberSwimmers=0;
}



public void cleanPool(int c)  throws
InterruptedException {
clean.acquire();
System.out.println(c+" is cleaning pool");
clean.release();
}
```

```
public void swim(int s)  throws InterruptedException
{
swim.acquire();
numberSwimmers+=1;
if (numberSwimmers==1) clean.acquire();
swim.release();


System.out.println(s+" is swimming happily");
swim.acquire();
numberSwimmers-=1;
if (numberSwimmers==0) clean.release();
swim.release();
}

}
```

Starvation

Starvation is a situation where a thread is unable to gain regular access to shared resources and is unable to make progress.

- This happens when shared resources are made unavailable for long periods by "greedy" threads.
  - For example, suppose an object provides a synchronized method that often takes a long time to return.
  - If one thread invokes this method frequently, other threads that also need frequent synchronized access to the same object will often be blocked.

Solution for ***First*** **variation** **(*reader-preference)*** – no reader kept waiting unless writer has permission to use shared object – has potential for starvation of Writers.

The Little Book of Semaphores
Allen B. Downey
Version 2.2.1

Do Morning Quiz 10 now

- Password: writer

Readers-Writers pool cleaning with semaphores



- How would starvation occur in this example?

```
import java.util.concurrent.Semaphore;


public class Pool {
private Semaphore swim;
private Semaphore clean;
private int numberSwimmers;


Pool() {
    this.swim = new Semaphore(1);
    this.clean = new Semaphore(1);
this.numberSwimmers=0;
}



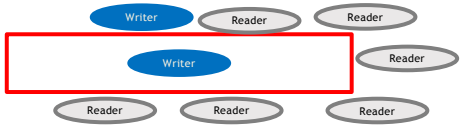
public void cleanPool(int c)  throws
InterruptedException {
clean.acquire();
System.out.println(c+" is cleaning pool");
clean.release();
}
```

```
public void swim(int s)  throws InterruptedException
{
swim.acquire();
numberSwimmers+=1;
if (numberSwimmers==1) clean.acquire();
swim.release();


System.out.println(s+" is swimming happily");
swim.acquire();
numberSwimmers-=1;
if (numberSwimmers==0) clean.release();
swim.release();
}

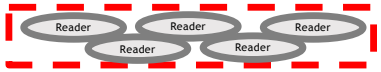
}
```

Readers -Writers problem solution with semaphores



Writer

Reader



Solution for *Second* variation - *writer-preference*?

Programming exercise

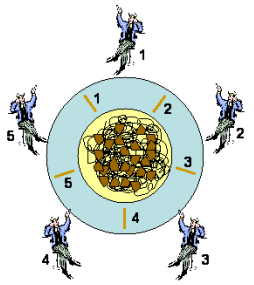




Java Marco-Polo game:

Barrier + rendezvous

Classical synchronization problems: Dining philosophers



Philosophers spend their lives alternating between thinking and eating

- Don't interact with their neighbors,
- Occasionally try to pick up 2 chopsticks (one at a time)

  to eat from bowl
  - Need both to eat, then release both when done

```
while True :
    think ()
    get_chopsticks ()
    eat ()
    put_chopsticks ()
```

Dining philosphers solution with semaphores

Philosopher

```
do {
     wait (chopstick[i] );
     wait (chopStick[ (i + 1) % 5] );

                    //  eat

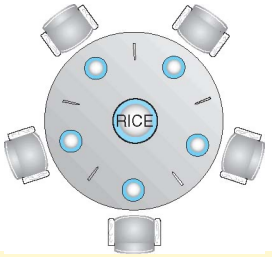
     signal (chopstick[i] );
     signal (chopstick[ (i + 1) % 5] );

                        //  think

   } while (true);
```

What is the problem with this algorithm?

Classical problems of synchronization:



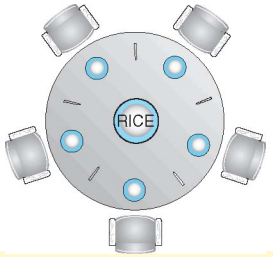
Dining philosophers

- See Java example (uploaded on Amathuba).
- Exercise: fix this so deadlock not possible.

Do Morning Quiz 11 now

- Password: deadlock

Classical problems of synchronization:



Dining philosophers

Standard solutions prone to deadlock.

Several solutions:

- Allow **at most 4 philosophers** to be sitting simultaneously at the table (**with 5 chopsticks**).
  - (deadlock then impossible)
- Allow a philosopher to pick up the chopsticks only if both are available (picking up must be done in a **critical section**).
- Use an **asymmetric solution** -- an odd-numbered philosopher picks up first the left chopstick and then the right chopstick. Even-numbered philosopher picks up first the right chopstick and then the left chopstick.

Also issue of starvation…?

```
while True :
    think ()
    get_chopsticks ()
    eat ()
    put_chopsticks ()
```

Starvation

Many dining-philosophers solutions can suffer from starvation even if they avoid deadlock.

- Deadlock freedom only guarantees that some philosopher can eat;
- starvation freedom requires that every hungry philosopher eventually gets a chance to eat.

To avoid starvation, the solution needs some notion of **fairness** e.g

- queueing philosophers in order of request
- fair semaphores

Then a hungry philosopher cannot be postponed forever.

Deadly embrace Deadlock with semaphores

- E.g. Let *S* and *Q* be two semaphores initialized to 1

P_0	P_1
`wait(S);`	`wait(Q);`
`wait(Q);`	`wait(S);`
`...`	`...`
`signal(S);`	
`signal(Q);`	`signal(S);`
	`signal(Q);`

Problems with semaphores

Semaphores can be very useful for solving concurrency problems, *but only if programmers use them properly* - **you have to be very careful!**

Timing errors from incorrect use of semaphore operations:

- inverting order

```
wait (mutex)

      ….
signal (mutex)
```

critical section can be violated.

- replacing signal with wait (accidentally)

```
wait (mutex)
….
wait (mutex)
```

Will result in **deadlock.**

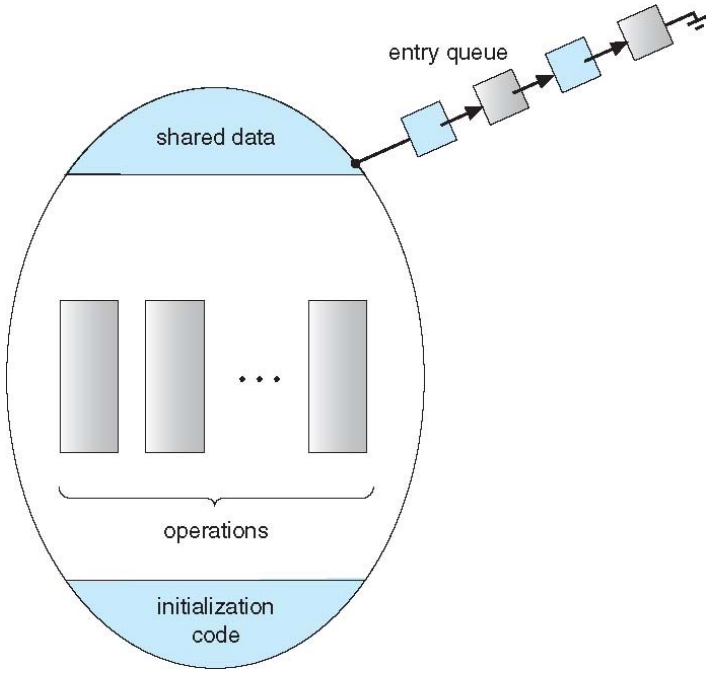
- Omitting `wait (mutex)` or `signal (mutex)` (or both) - mutual exclusion violation or deadlock

To deal with this created high-level language constructs for synchronization…

Monitors

High-level abstraction that provides a convenient and effective mechanism for process synchronization

- *Abstract data type*
  - internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time




```
monitor monitor-name
{
        // shared variable declarations
        procedure P1 (…) { …. }

        procedure Pn (…) {……}

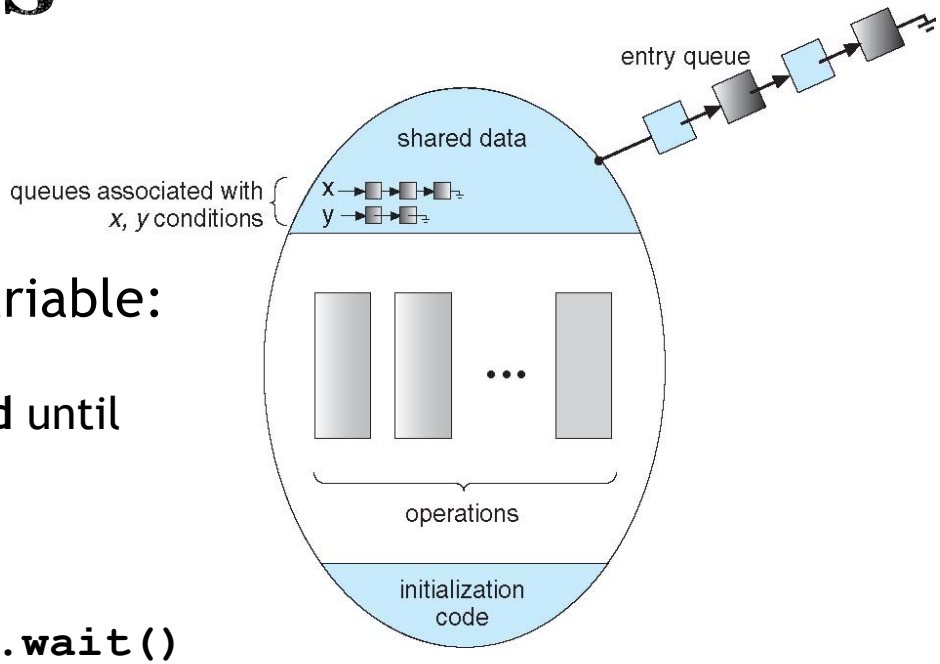
         initialization code (…) { … }
        }
}
```

A monitor is essentially a class in which all data is private, and where **only one method** within any given monitor object **may be active** at the same time.

But not powerful enough to model some synchronization schemes….

Condition Variables

`condition x;`



Two operations are allowed on a condition variable:

`x.wait()`

- a process that invokes the operation is **suspended** until

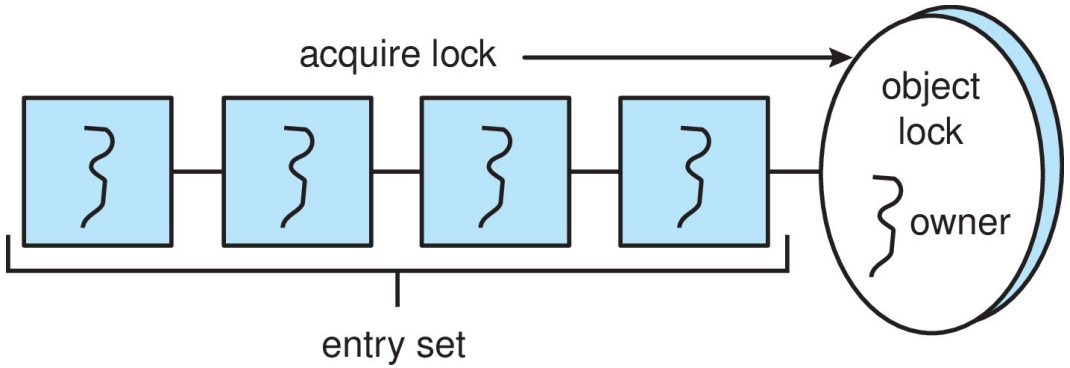
`x.signal()`

`x.signal()`

- resumes **one of processes** (if any) that invoked **`x.wait()`**
  - If no **`x.wait()`** on the variable, then it has no effect on the variable

Java Monitors

- Every Java object has associated with it a single lock.
- If a method is declared as **`synchronized`**, a calling thread must own the lock for the object.
- If the lock is owned by another thread, the calling thread must wait for the lock until it is released.
- Locks are released when the owning thread exits the **`synchronized`** method.
- A thread that tries to acquire an unavailable lock is placed in the object's **entry set**:



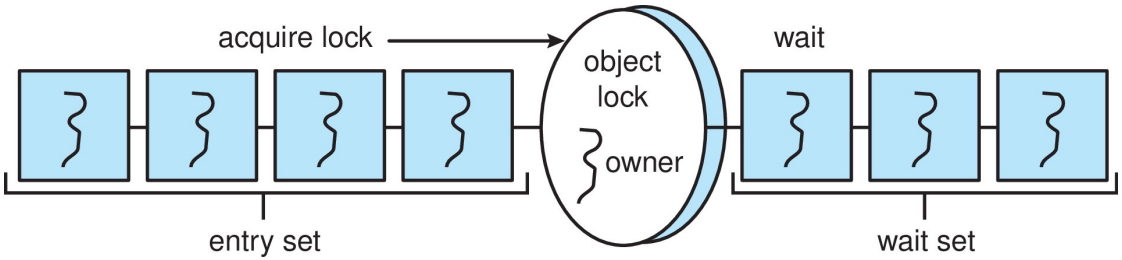


Java Synchronization

Similarly, each object also has a **wait set** – it is also a condition variable

When a thread calls `wait()` on an object:

1. It **releases the lock** for the object
2. The state of the thread is set to **blocked**
3. The thread is placed in the **wait set** for the object





Java Synchronization

A thread typically calls wait() when it is waiting for a condition to become true.

- always check the condition in a loop, not just an if, when using wait()

How does a thread get notified?

When a thread calls **`notify()`**:

1. An arbitrary thread T is selected from the wait set
2. T is moved from the wait set to the entry set
3. Set the state of T from blocked to runnable.

T can now compete for the lock to check if the condition it was waiting for is now true.



Java monitor pattern

Every object is both **a condition variable** (wait() and notify()) and a **lock/monitor**.

- Lock is implied with `synchronized` keyword
- An object following the Java Monitor pattern encapsulates all its mutable state and guards it with the object's **own intrinsic lock**
  - Used by many library classes:
    - Vector
    - HashTable
- Advantage is that it is simple
- Disadvantages is poor concurrency
  - thread safety achieved by serializing all access to collection's state -only process one request at a time

Producer-Consumer with monitor




```
public class Buffer {
private int[] rack;
private int in, out;
private int RACK_SIZE;
private int counter;
Buffer(int rackSize) {
    this.rack = new int[rackSize]; //
    this.in = 0;
    this.out =0;
    this.RACK_SIZE=rackSize;
    this.counter=0;
}
public synchronized void addWidget(int widget_id)  throws InterruptedException {
        while (counter>=RACK_SIZE) {this.wait();}
        rack[in]=widget_id; // add widget to buffer
        in=(in+1)%RACK_SIZE; //circular buffer
        counter+=1;
        System.out.println("added widget #"+widget_id+" to buffer.");
        this.notifyAll();
}
public synchronized int removeWidget() throws InterruptedException {
        while (counter==0) {this.wait();}
        int next=rack[out];
        out = (out + 1) % RACK_SIZE; //circular buffer
        counter-=1;
        System.out.println("removed widget #"+next+" from buffer.");
        this.notifyAll();
        return next;
}}
```

Need `while` loop because of

- spurious wakeups
- another thread may have changed the state before the awakened thread reacquires the lock

Java Synchronization

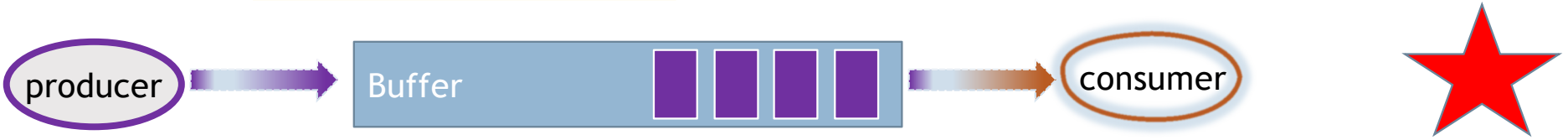
Java provides rich set of synchronization features:

- Atomic variables
- Semaphores
- Reentrant locks
- Condition variables
- Java monitors
- **Concurrent collections -**
  - e.g BlockingQueue defines a first-in-first-out data structure that blocks or times out when you attempt to add to a full queue, or retrieve from an empty queue.

In practice, prefer well-tested concurrent data structures such as BlockingQueue when they fit the problem, rather than implementing synchronization manually.



Producer-Consumer with BlockingQueue




```
private BlockingQueue<Integer> shared_rack;
```

```
shared_rack.put(counter);
```

```
counter =shared_rack.take();
```

Linux Synchronization

Linux is a fully preemptive kernel* - so has potential race conditions

- Linux provides:
  - Semaphores
  - **atomic integers**
    - **`atomic_t`** is the type for atomic integer
    - All mathematical operations performed without interruption
  - Mutex locks
  - spinlocks
  - reader-writer versions of both

Atomic Operation	*Effect*
`atomic_set(&counter,5);`	`counter = 5`
`atomic_add(10,&counter);`	`counter = counter + 10`
`atomic_sub(4,&counter);`	`counter = counter - 4`
`atomic_inc(&counter);`	`counter = counter + 1`
`value = atomic_read(&counter);`	`value = 12`



Programming exercise





Java Marco-Polo game:

Barrier + rendezvous

> “Simplicity is prerequisite for reliability.”
>
> — **Edsgar W. Dijkstra**